Graphical User Interfaces

Contents

How to create GUIs in Python

Why do we need GUIs?

- As we learned when programming in python we generate scripts we can than hand over to the Interpreter
- Since we only have these scripts, most Python programs are console Programs without a graphical user interface (GUI)
- Furthermore we always need an Interpreter or a terminal to start our programs, which can be a bit troublesome
- If you want programs to be used by others making changes to certain variables at each start might be unrealistic and a GUI might make this process easier
- Therefore there are several modules that allow us to generate GUIs in Python

Modules to create GUIs [1]

- PyGUI [2]
- Wax [3]
- Pyforms [4]
- PySimpleGUI [5]
- Libavg [6]
- Kivy [7]
- PySide2 [8]
- wxPython [9]
- PyQt5 [10]
- Tkinter [11]

What is tkinter?

Tkinter [11,12]

- Stands for ToolKit Interface
- A module with graphical elements that was designed for generating graphical user interfaces with the help of Python scripts
- Originally it was no standard module even though it was written by Steen Lumholt and Guido van Rossum
- Later it was included in the standard Python Library and could be seen as Python‘s de facto standard GUI

- `import tkinter as tk`

What are Widgets?

Widgets

- Definition: A widget is a graphical component on the screen (button, text label, drop down menu, scroll bar, image, etc…)
- GUIs are generated by combining different widgets and arranging them on the screen
- In General we define them as variables during the construction as it‘s shown down below
- Nearly each Widget needs a parent, another Widget on which we can place it

Default form of contructors in Tkinter:

widget = <widgetname>(parent,options…)

What are options?

- We can adjust the look of our GUI and Widgets with the help of options, that allow us to modify the look of our Widgets
- In most cases we define these options when constructing our widgets and we always have to use the following syntax:
  - optionname = value
- The optionname is fixed and we have some standard options that are usable for all widget as well as some options that are specific for certain widgets
- The value than determines the look of our widget, for example the color of text on our widget
- If we don‘t specify an option tkinter takes the default values from the system

Some standard options

- **background or bg:** color of the background of an inactive widget
- **foreground or fg:** color of the foreground of an inactive widget (e.g. the text on a button)
- **height:** Height of a widget (text units for widgets containing text and pixel for images)
- **width:** Width of a widget (text units for widgets containing text and pixel for images)
- **state:** is the widget clickable or not (“normal”, “disabled”)

Reading out values of options of Widgets

- tkinter allows us to read out the values of options we assigned to a widget by using getter() methods
- For general options of widgets we have to use the method cget() and hand over the name of the options in form of a string as argument to it
- Down below you can see an example to read out the text written on a Button:

Get values of widget options:

```
Widget.cget(option)
```

```
Button1.cget(„text“)
```

Changing options after construction

- I mentioned that we mainly assign value to our options when constructing our widgets
- However tkinter allows us to dynamically change the values of these options, for example to change the text on a widget after a button click
- For widget options we use the setter method config() to do this
- This method takes multiple options in the form of optionname=value

Set values of widget options:

```
Widget.config(option = value)
```

```
Button1.config(width = 12)
```

The most important Widget

What is a parent?

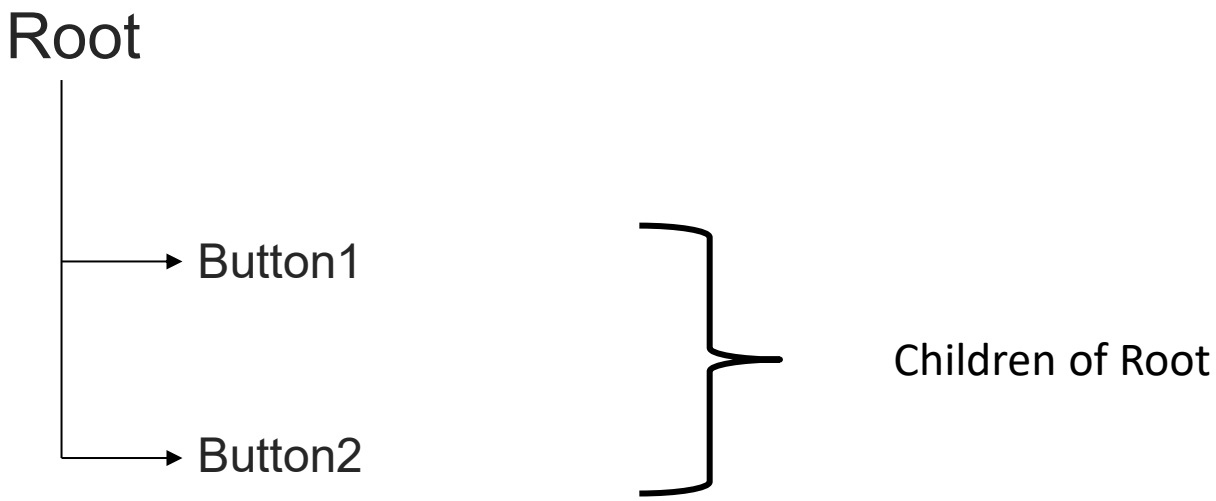
- When introducing widgets I mentioned that all widgets need a parent when we construct them
- A parent is a widget on which we can place other widgets and in most cases this is the window we want to generate as GUI
- In tkinter the window is called Tk and is constructed as follows:

```
root = tk.Tk()
```

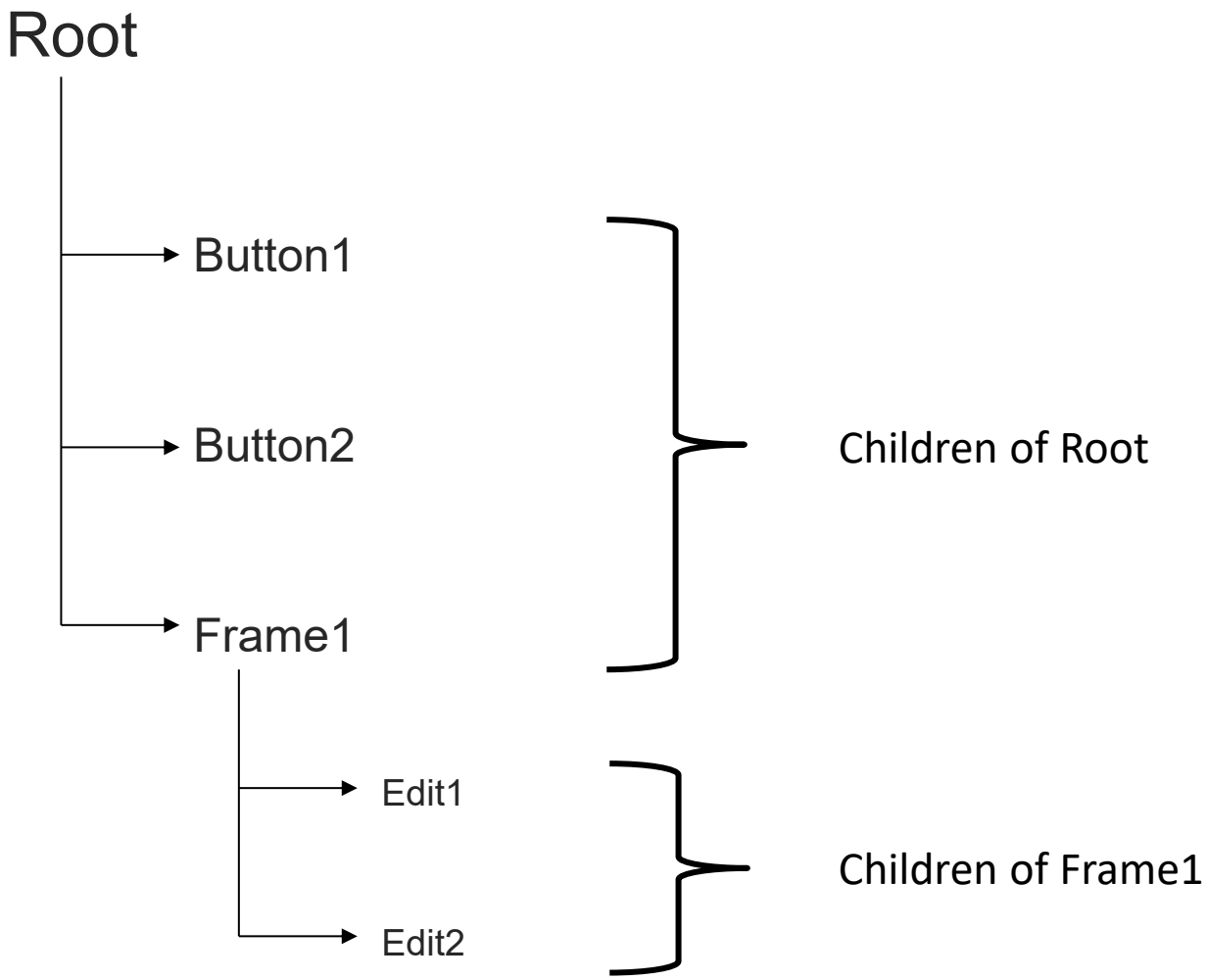
- Please keep in mind that you are free to choose how to name this widget, I used the name root, since it‘s good programming practice and in some documentations of tkinter you will also find that name
- Other commonly used names are master or top or window

Root or Window

- This widget that contains nearly all other widgets and is thus the parent of them
- Widgets generate a tree-like structure



Other widgets can also be parents



Your first Widget – The Label

Label

- Labels are used to display text on your GUI and nothing more
- They are the most simple widgets and the most important option of them is:
  - text: text that should be displayed
- Labels are created with the function tk.Label()
- As arguments we have to hand over a parent of our label first, in most cases the root, and afterwards we can define the remaining options
- We can construct a simple label that displays the text “Hello World” as follows:

```
label1 = tk.Label(root, text= "Hello World!")
```

Insert: Placement of widgets

- One thing we have to keep in mind when working with tkinter is that only constructing our widget is NOT enough to be able to see it on our window
- We always need to use a so-called geometry manager to put our widget onto our GUI
- In tkinter we have three different managers available and we will talk about them more in depth near the end of the course
- For now we will use the simple manager pack() to place our widget from top to bottom on our GUI
- We can do this by invoking the pack() method on our widgets:

```
label1.pack()
```

Insert: Starting our GUI

- After we created all our widgets and put them onto our GUI with the help of a geometry manager we now need to start our window
- We do this by invoking the mainloop() method on our main window, in most cases named root, as follows:

```
root.mainloop()
```

- When Python encounters this line, the interpreter stays at it for as long as our window is opened and now we can execute functions with the help of buttons for example
- The neat thing about this is, that even if our function produces an error our GUI and our program won‘t stop

Exercise 04: Creating Labels

- Create a GUI that contains three different Labels
- Please try to use texts with a biological or biochemical background, we will need these labels later

Insert: Using Fonts I

- Generally tkinter widgets use the fonts of the system, but we are able to define our own fonts
- When we want to do so we can use the tkinter.font module
- We have to import it like this:

```
import tkinter.font as font
```

- Now we can define our own fonts simply by using:

```
myFont2 = font.Font(family="Times",weight="bold",size=10)
```

- Please keep in mind that you will only see changes in your fonts if the corresponding font family is known to your operating system

Insert: Using Fonts II [13]

- When creating your own fonts you can specify six different options for it:

1. family ➔ the font family name as a string
2. size ➔ The font height as an integer in points
3. weight ➔ "bold" for boldface or "normal" for standard
4. slant ➔ "italic" for italic or "roman„ for standard
5. underline ➔ 1 for underlined text or 0 for normal
6. overstrike ➔ 1 for overstruck text or 0 for normal

- If you don‘t specify a certain option, than the default value is taken instead

Something to click on

Button

- On GUIs many functions are started with the click on a Button
- These widgets can be associated with a specific function we defined to execute some calculations for example
- Aside of the text attribute we already saw for Labels, the most important option of a button is:
  - command: a function that should be run if the button is clicked

- We can create a button that shows the text “Click me!” and executes the destroy method on our root as follows:

```
button1 = tk.Button(root, text=„Click me!“, command=root.destroy)
```

Insert: root.destroy

- On the previous slide we saw the destroy method used on our root
- This method allows us to delete or remove the widget on which we use it
- That means the command root.destroy, deletes our window and thus closes our GUI
- So you can use this function as an alternative closing button on your GUI for example
- You can also use this function on other widgets to permanentely remove them from your GUI

Associating functions with Buttons

- When creating buttons we can associate functions with them by using the option command
- When doing so we need to be aware of three things:

1. The function we associate with our Button can NOT have any parameters in its signature
2. We can NOT hand back any values from our function via a return statement
3. We only write the name of our function after the option command WITHOUT brackets

Exercise 05: Using Buttons

- Use your GUI from Exercise 04 and add two Buttons to it
- Each Button should change the text in one of your three pre-defined labels

Getting Values from the user

VarVars I

- We can construct different widgets in tkinter that allow us to get an input from the user, be it as text, a number or in form of check boxes
- Whenever we use a widget that can be used for this purpose we have to associate it with a variable that can be used to read out the value from our widget
- Unfortunately we can NOT use normal variables and have to use specific variables that are unique to tkinter
- These variables are called VarVars and there are four different types of them
- Please keep in mind that you need to define these VarVars BEFORE you can associate them with a widget afterwards

VarVars II

- For text we have StringVar:

```
text1 = tk.StringVar()
```

- For integers we have IntVar():

```
number1 = tk.IntVar()
```

- For floating point numbers we have DoubleVar():

```
float1 = tk.DoubleVar()
```

- And for Bool (True or False) we have BooleanVar():

```
truth1 = tk.BooleanVar()
```

- After declaring them like this, they do NOT contain a value yet, we just created the container in which we or the user can later put in the corresponding value

VarVars III

- After we declared / defined our VarVar we can give it a value to carry with the help of a setter method
- For VarVars this method is simple called set and in brackets we write the new value we want assign to our VarVar

```
text1.set("this is a text")
```

- If we want to read out the value of a VarVar we ALWAYS need a getter method to access the value saved in it
- For VarVars this method is called simply get() and we can use it to declare normal variables as well

```
value_text = text1.get()
```

VarVars IV

- An alternative to defining the VarVar first and than setting a value using the set() method is to directly specify a value during the definition of the VarVar
- For this you need the option value of the VarVar:

```
var1 = tk.StringVar(value="Python")
```

- In the example above the StringVar var1 gets assigned the value of „Python“ during its creation

Entry I

- This widget can be used for short inputs and outputs
- Always needs a VarVar that is associated with it, with the help of the option textvariable:

```
entry1 = tk.Entry(root, textvariable=input1)
```

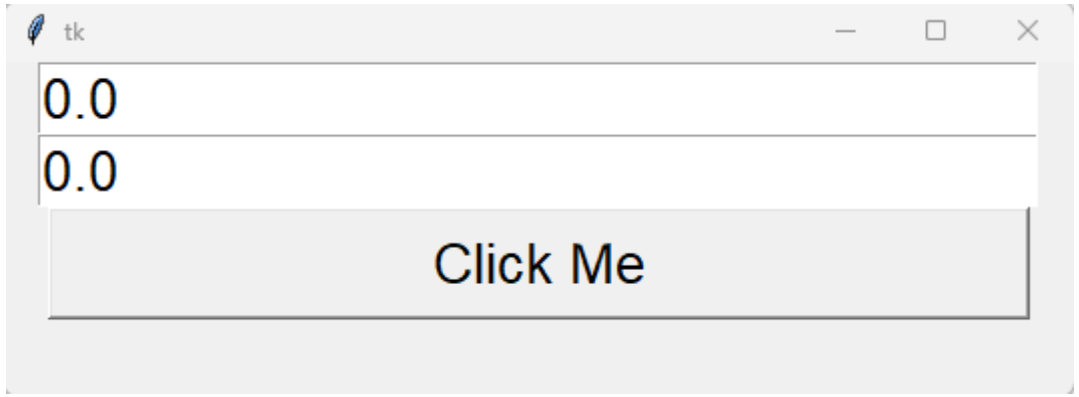
- The corresponding VarVar can be of any type and the look of our Entry changes depending on which type we choose

Entry II

- StringVar()

- IntVar()

- DoubleVar()



- BooleanVar()

Exercise 06: Entries

- Add two Entry fields to your GUI from Exercise 05 so that the user can input what information he wants to display in your changeable labels
- Don’t forget to adjust your functions for the Buttons
- Afterwards try your hands on changing other options as well

Other forms of inputs

User Inputs

- We can generate user inputs in many different ways and there are many different widgets that allow the user to specify inputs:
  - Checkbutton ➔ allows the user to hand over Boolean inputs
  - Radiobutton ➔ allows the user to choose one of multiple options
  - Scale ➔ allows the user to input numerical values in form of a scrollbar
  - Spinbox ➔ allows the user to input numerical values from a pre-defined range
- All of these widgets have advantages and disadvantages and you need to decide which you like best for the task your given

Checkbutton I

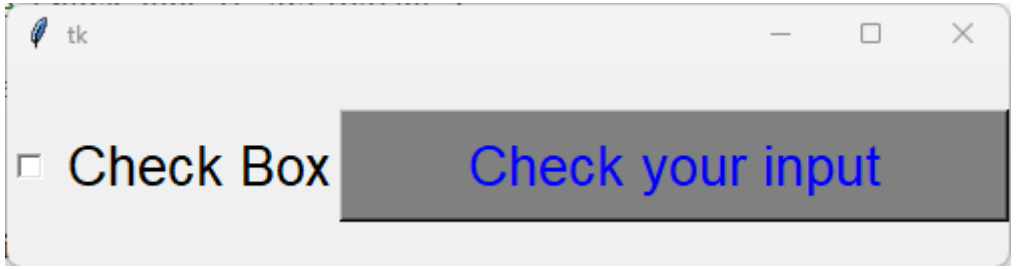
- We can allow the user to input Boolean values (True or False) with the help of a widget called Checkbutton
- Each Checkbutton needs to be associated with a DIFFERENT VarVar of type Boolean by using the option variable
- We can create a Checkbutton with the text “Check Box” shown on the right side of the button as follows:

```
checkValue = tk.BooleanVar()

checkValue.set(True)

check1 = tk.Checkbutton(root, text="Check Box", variable=checkValue)
```

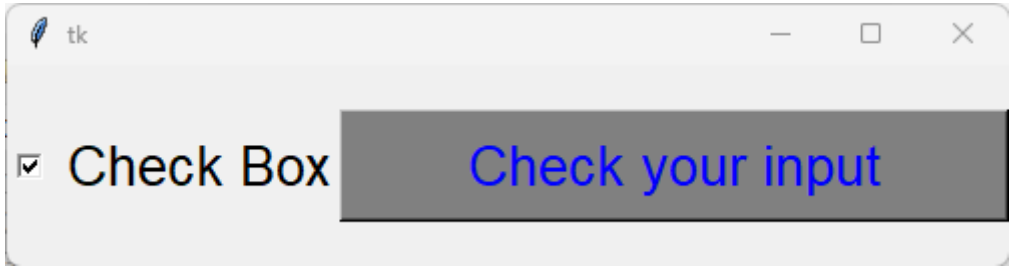
Checkbutton II

- Depending on the value we assing to the Boolean VarVar in our script our Checkbutton looks different on the start of our GUI

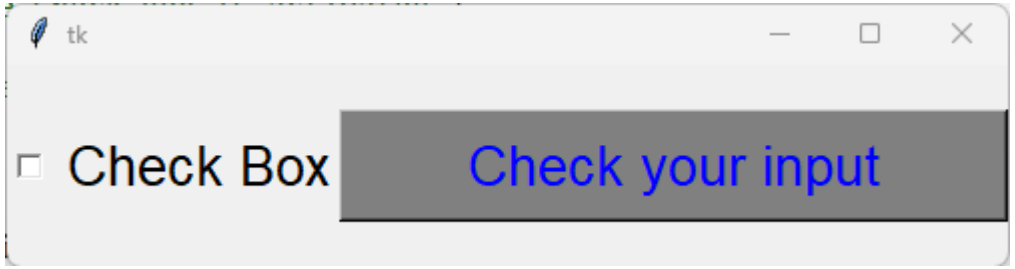
- without set()



- set(True)

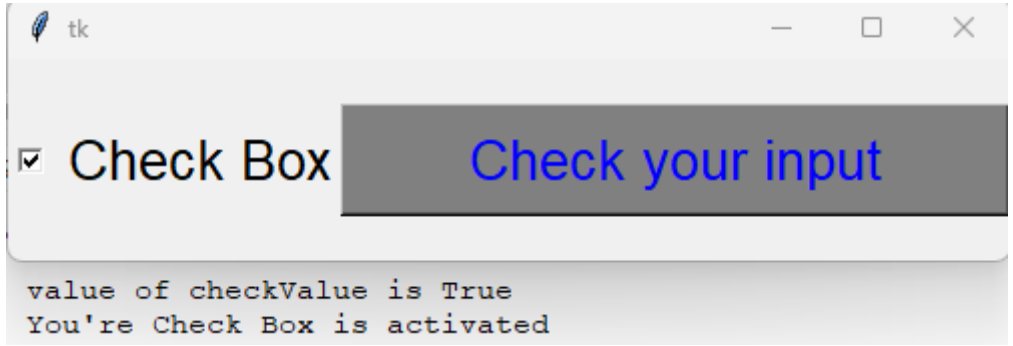


- set(False)

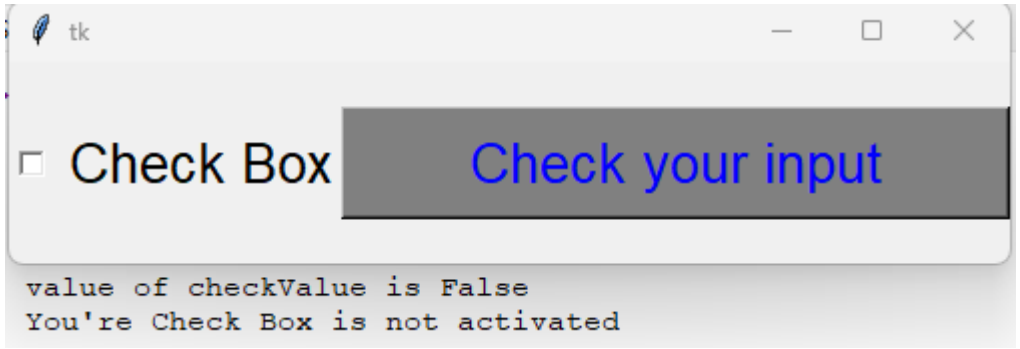


Checkbutton III

- When a checkbutton is activated the value of the corresponding VarVar is True



- When a checkbutton is deactivated the value of the corresponding VarVar is False



Exercise 07: Checks

- Create a GUI with:
  - 1 Button
  - 5 Labels
  - 5 Check Buttons
- On Button click the current status of the five Check Buttons should be displayed in the corresponding Label

Radiobutton I

- We can allow the user to choose between one of multiple options with the help of Radiobuttons
- These widgets are grouped together by associating the same VarVar to a group of them with the help of the option variable
- From such a group only one can be active at the same time
- Of course we can display text on the right side of this widget with the help of the option text, similar to how we did it with Checkbuttons

Radiobutton II

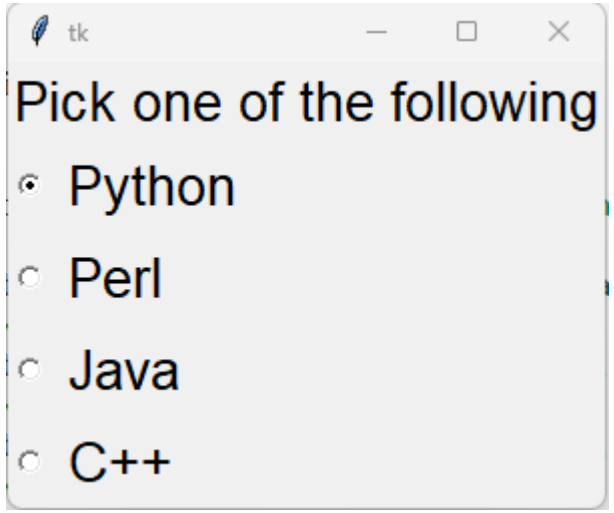
- When constructing these widgets we need to associate our VarVar with them using the option variable AND we need to specify which value should be assigned to this VarVar when the corresponding Radiobutton is active with the help of the option value:

```
var = tk.StringVar()

var.set("Java")

radio1 = tk.Radiobutton(root,text="Java",variable=var,value="Java")

radio2 = tk.Radiobutton(root,text="Perl",variable=var,value="Perl")
```

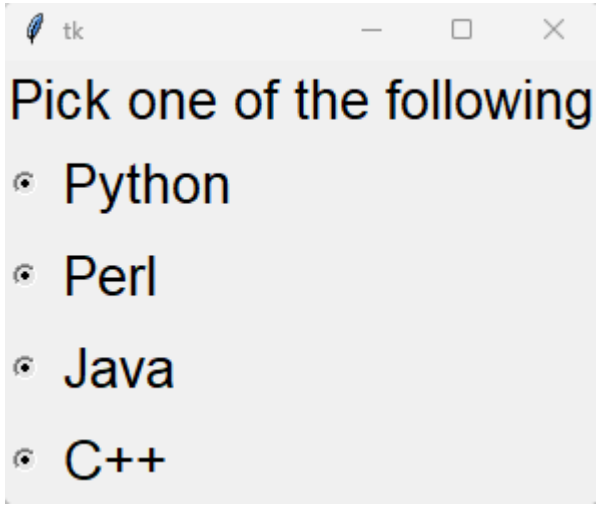
Radiobutton III

- Similar to Checkbuttons the look of these widgets at the start of our GUI depends on whether or not we assigned a value to our VarVar

- With assigned value



- Without assigned value



Exercise 08: Radio Buttons

- Create a GUIs with at least eight Radiobuttons
- Create a function that is coupled with the Radiobuttons

Scale I

- We can create widget in form of scroll bars that allow the user to input numerical values from a certain range we determine
- Scales work with VarVars of the types Integer or Float and allow us to not only specify a range but also a step size for the value the user can choose
- Furthermore they can be horizontal or vertical
- The widget we can use for this purpose is called Scale and we can construct it as follows:

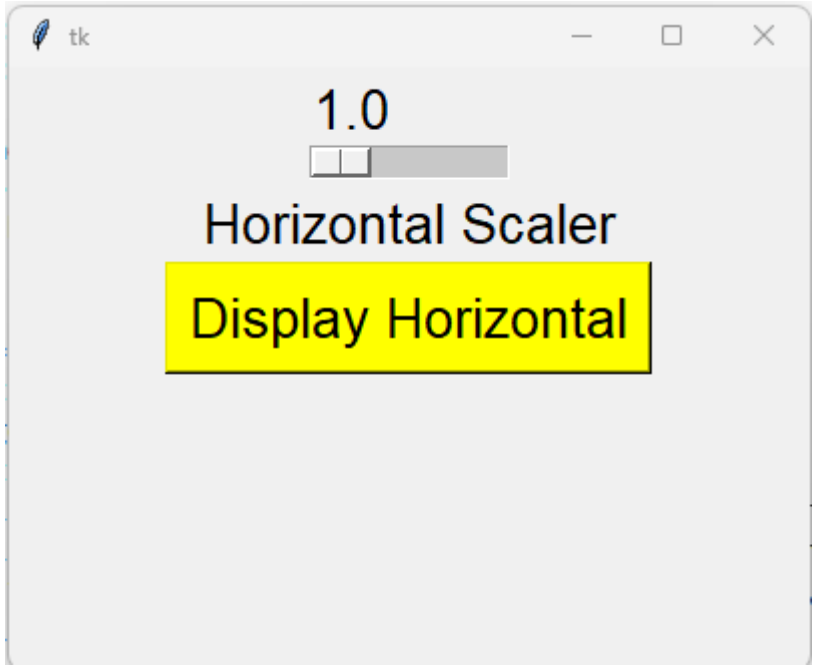
```
v1 = tk.DoubleVar()

scale1 = tk.Scale(root,variable=v1,from_=1,to=10,resolution=0.2)
```

Scale II

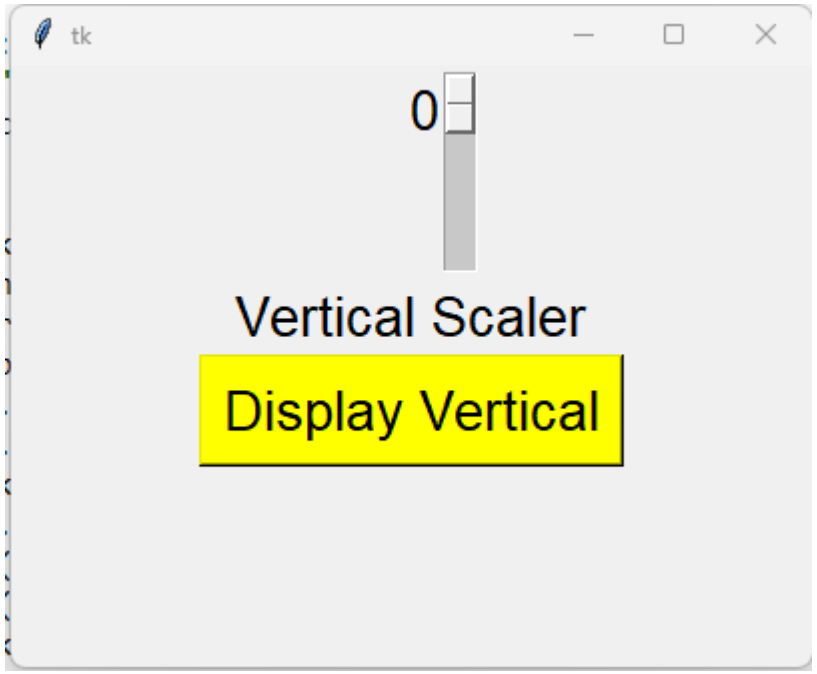
- The options in the constructor have the following meaning:
  - variable — is used to associate our VarVar with the widget
  - from_ — is the included starting point of our range
  - to — is the included end point of our range
  - orient — determines whether our bar is horizontal or vertical (next slide), if not specified we get a vertical one
  - resolution — is the stepsize of our range, per default the resolution is 1

Scale III

- orient="horizontal"



- orient="vertical"



Exercise 09: Two Scales

- Create a GUI with two Scales
  - One horizontal
  - One vertical
- Display the chosen values of both scales in a Label on the click on a Button
  - Similiar to a coordinate system

Spinbox I

- Instead of a scale we can also use a Spinbox to allow inputs in a certain range
- The advantages of Spinboxes over scales are:
  - They are more compact than scales
  - They also allow the input of strings from a pre defined list of values
- Unlike other widgets they are NOT associated with a VarVar and instead we read out the value directly from our Spinbox using the getter method get() on our widget:

```
spinbox1.get()
```

Spinbox II

- When constructing a Spinbox we have two options, we can either hand over the starting point and end point of a range of numerical values similar to how we did when constructing Scales:

```
spinbox1 = tk.Spinbox(root, from_=0, to=10)
```

- Here we can also define a stepsize by adding the option increment to our constructor:

```
spinbox1 = tk.Spinbox(root, from_=0, to=10,increment=0.5)
```

Spinbox III

- Per default the stepsize is 1 and the values we read out are of type integer
  - As long as the increment is an integer value the values of our Spinbox are also of type Integer
  - The same is true for float values that represent whole numbers, e.g. 2.0
  - If the increment is a floating point value the values of the Spinbox are also of type float
  - Furthermore the amount of decimals of the increment determines the amount of decimals of the values, e.g. an increment of 0.5 leads to values having only 1 decimal
  - Be aware that tailing zeros, e.g. 0.50 or 1.0 are ignored

Spinbox IV

- As mentioned we can also construct a Spinbox with the help of a list of string elements to allow the user to pick from these values
- In this case we need to define such a list and than associate it with our Spinbox by using the option values:

```
text_values = ["Apple", "Banana", "Cabbage", "Beans"]

spinbox1 = tk.Spinbox(root,values=text_values)
```

- Please be aware that the option increment has no effect whatsoever when constructing a spinbox like this, so we just leave it out here

Exercise 10: Spin right round

- Create a GUI that offers the user three inputs via Spinboxes
- Display the result of each input in a separate label
- Use one spinbox with a float increment
- Use at least one in combination with a list of strings

Where to put what?

Geometry Manager

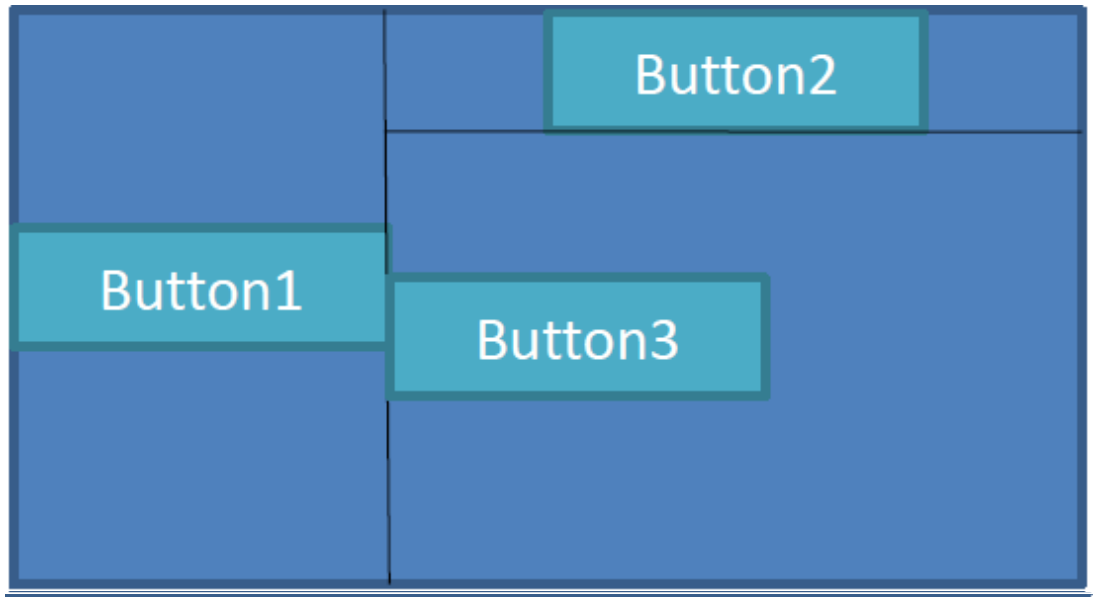
- While Tkinter compiles the GUI it has to arrange the widgets we constructed
- There are WYSIWYG (What you see is what you get [14]) Gui modules – Tkinter unfortunately isn‘t one of them
- Therefore we need to place our widgets with the help of geometry managers on our GUI
- This is done by invoking the corresponding geometry manager on our Widget
- There are three main managers:
  - pack
  - place
  - grid

Pack()

- Places the widgets in the free room of the parent with which the widget is associated, in most cases this is our root / window
- We can can put widgets only at the following specific places:
  - Left
  - Right
  - Top
  - Bottom
- After placing a Widget left or right the whole space above and beneath our widget is blocked
- After placing a Widget top or bottom the space aside our widget is blocked

Illustration of Pack

- Button1.pack(side=„LEFT“)
- Button2.pack(side=„TOP“)
- Button3.pack(side=„LEFT“)



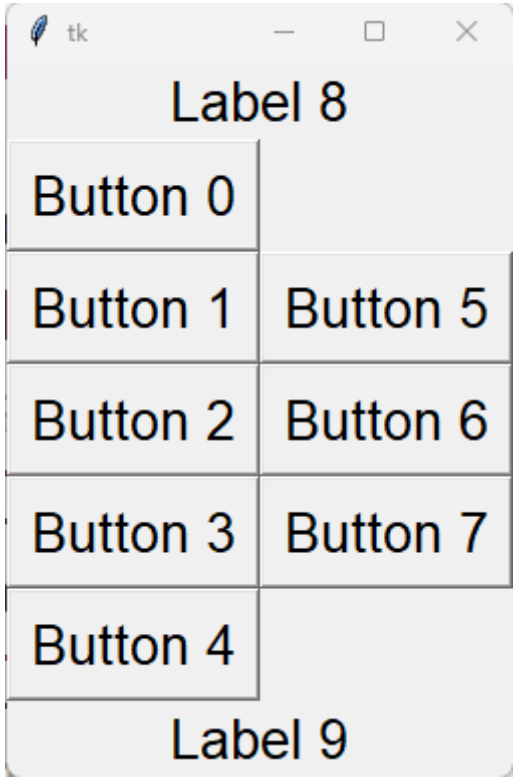
Last remarks about pack

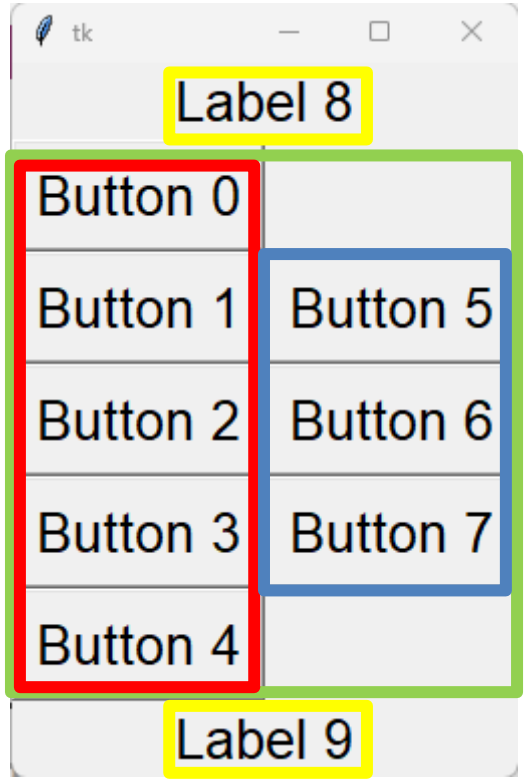
- Pack is best used to line widgets side by side or top to bottom
- But not both at the same time!
- Frames are needed to do more complicated layouts

Frame I

- We can use a widget called Frame to generate more complex layouts, since they can be the parent of other widget similar to our window
- We can group our widgets to be displayed together by putting them in the same frame, that means our widgets have the same frame as parent
- The parent for our frame can be the root but also other frames
- Constructor for frames:
  - frame1 = tk.Frame(parent)
- Every frame has to be packed just like any other widget after the construction

Frame II


tk
Label 8
Button 0
Button 1
Button 5
Button 2
Button 6
Button 3
Button 7
Button 4
Label 9

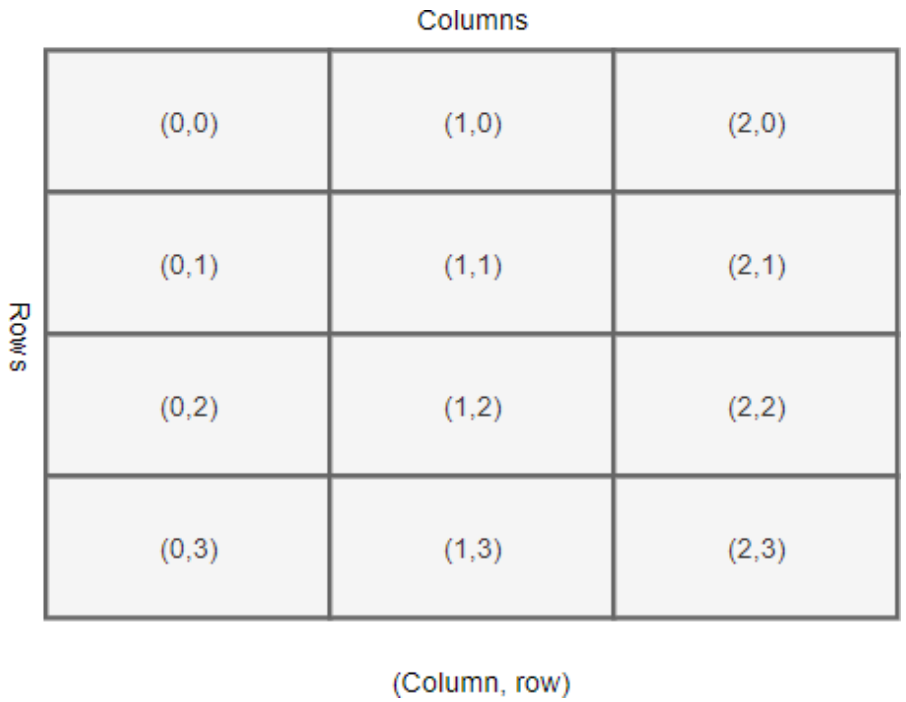

tk
Label 8
Button 0
Button 1
Button 5
Button 2
Button 6
Button 3
Button 7
Button 4
Label 9

Exercise 11: Framing

- Create a GUI that contains the following:
  - 4 Frames
  - 8 Buttons
  - 4 Labels
  - 5 Checkboxes
- The buttons should be placed in two Frames, the Labels in one Frame and the Checkbuttons in the last Frame
- Place the Frames top to bottom on your GUI and the Widgets inside the Frames left to right

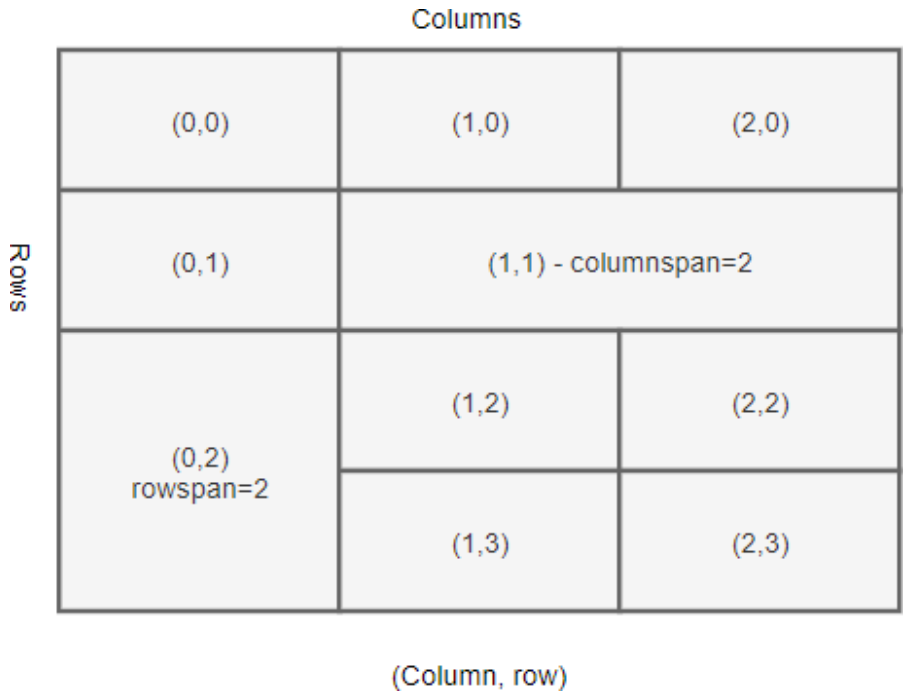
Grid I [14]

- Uses the concept of rows and columns to arange widgets
- Each field of the grid is accessible through the indices of the rows and columns
- Indices can have gaps and don‘t have to start at 0



Grid II [14]

- Each field can contain only one widget
- To add mutliple widgets in one field you have to use a Frame again
- Fields can span multiple rows or columns as well



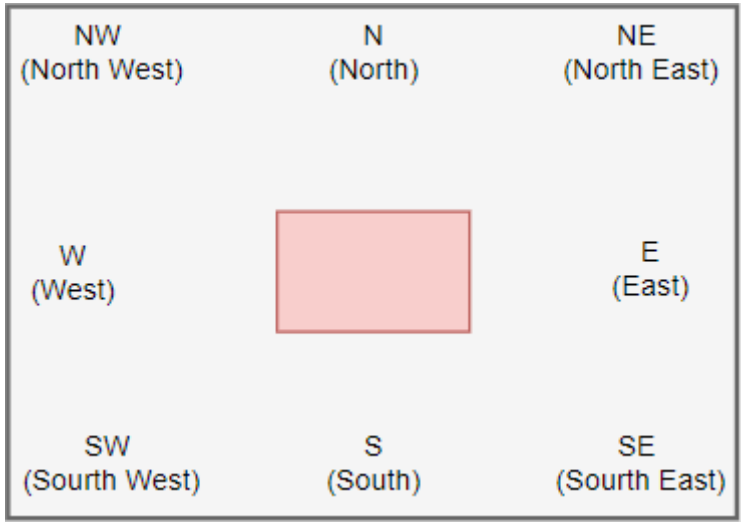
Grid III [14]

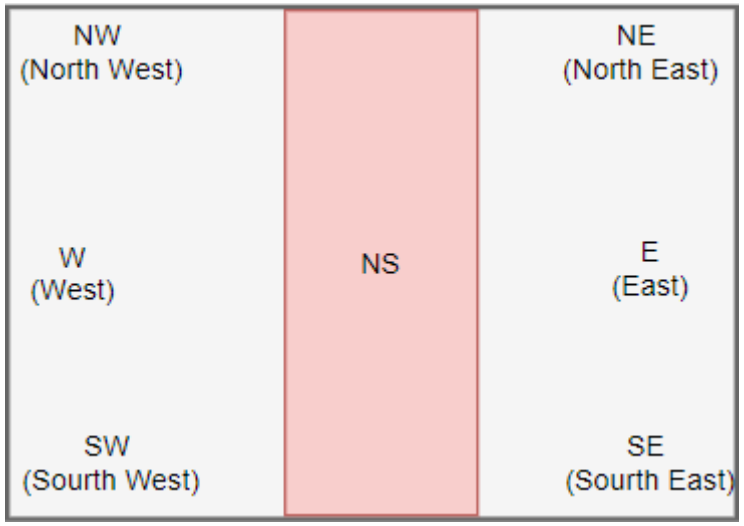
- To position the widget on the grid you have to use the grid method: widget.grid()
- Inside the brackets of grid() we have to specify the options row and column to tell grid where to place the widget on our GUI
- All other options down below are optional

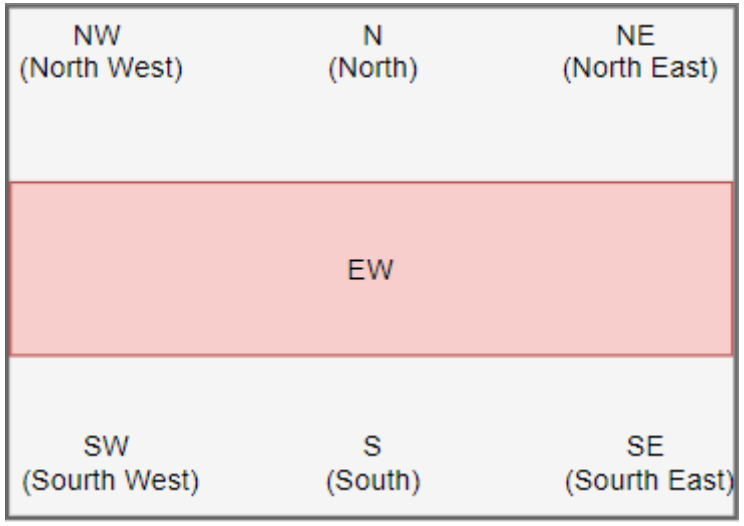
Parameter	Meaning
column	The column index where you want to place the widget.
row	The row index where you want to place the widget.
rowspan	Set the number of adjacent rows that the widget can span.
columnspan	Set the number of adjacent columns that the widget can span.
sticky	If the cell is large than the widget, the stick option specifies which side the widget should stick to and how to distribute any extra space within the cell that is not taken up by the widget at its original size.
padx / pady	Border around our grid in pixel for horizontal (x) and vertical (y)

Sticky [14]

- By default if the field is larger than the widget, the widget is placed in the center
- You can adjust this placement with the sticky method
- You can either use strings like “w” to achieve this or use the tkinter versions like tk.W







Exercise 12: Using Grid

- Create a GUI with Grid
- Include at least 10 different widgets:
  - One Button is a must
  - The Button should have a function assigned to him

Place I

- The third geometry manager available to us
- Allows us to place Widgets pixel exactly on our GUI
- You are able to define the size of your widgets in pixels and not text units
- Allows us to adjust placement and size of widgets according to the window size
- All values of place are seen as relation between the top left corners of our widget and its parent (the root or a Frame)

Place II

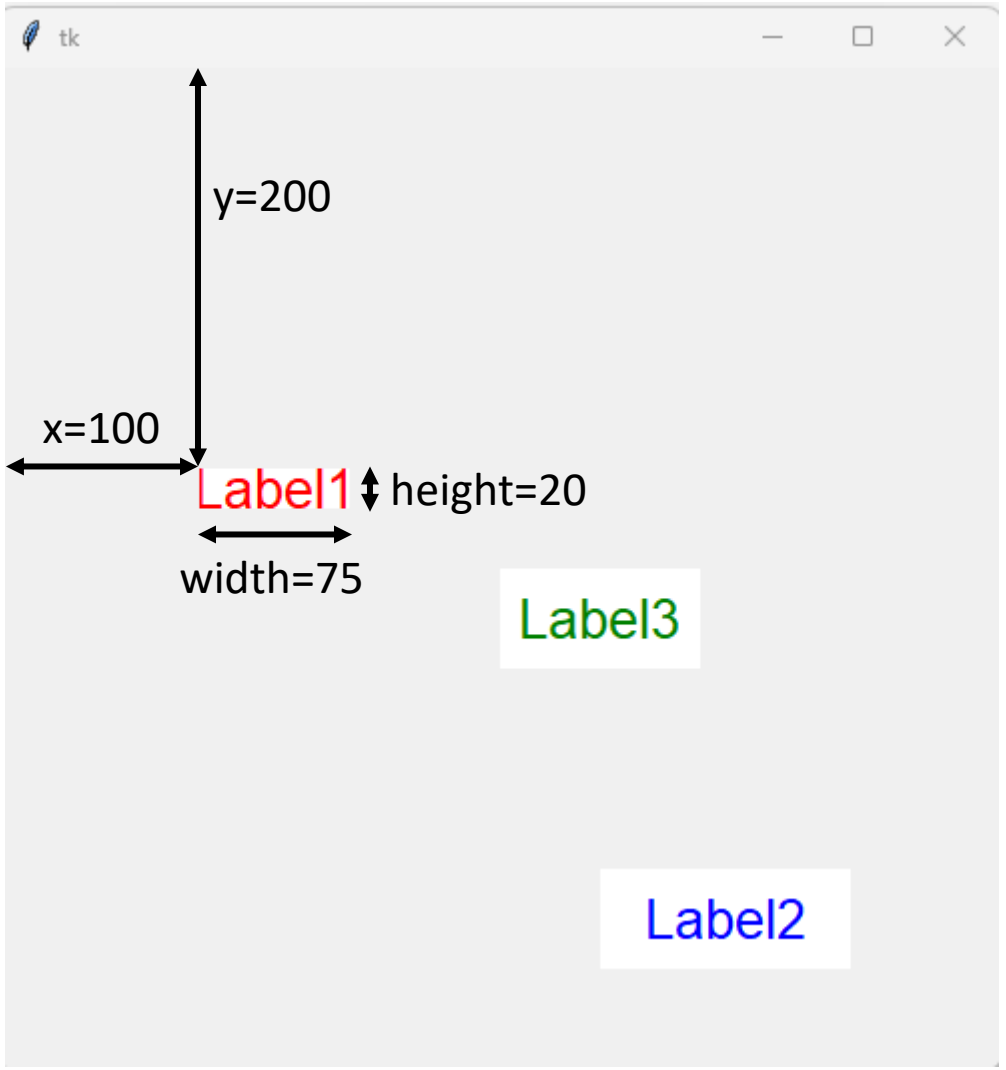
- We can put our widgets pixel exactly into our GUI by using place on our widget and specifying the coordinates where we want to place our widget with the options x and y in combination with integer values:

```
label1.place(width=75,height=20,x=100,y=200)
```

- These values refer to the placement of the top left corner of our widget in relation to the top left corner of the window
- That means the top left corner of label1 is 100 pixels to the right and 200 pixels to the bottom when looking at the top left corner of our window (next slide)
- Furthermore we can define the exact size of our widget in pixels with the optional options width and height

Place III

- Dimensions of place:
  - `width=75`
  - `height=20`
  - `x=100`
  - `y=200`



Place IV

- We can also place our widgets relative to our window by using the options relx and rely in combination with values between 0.0 and 1.0:

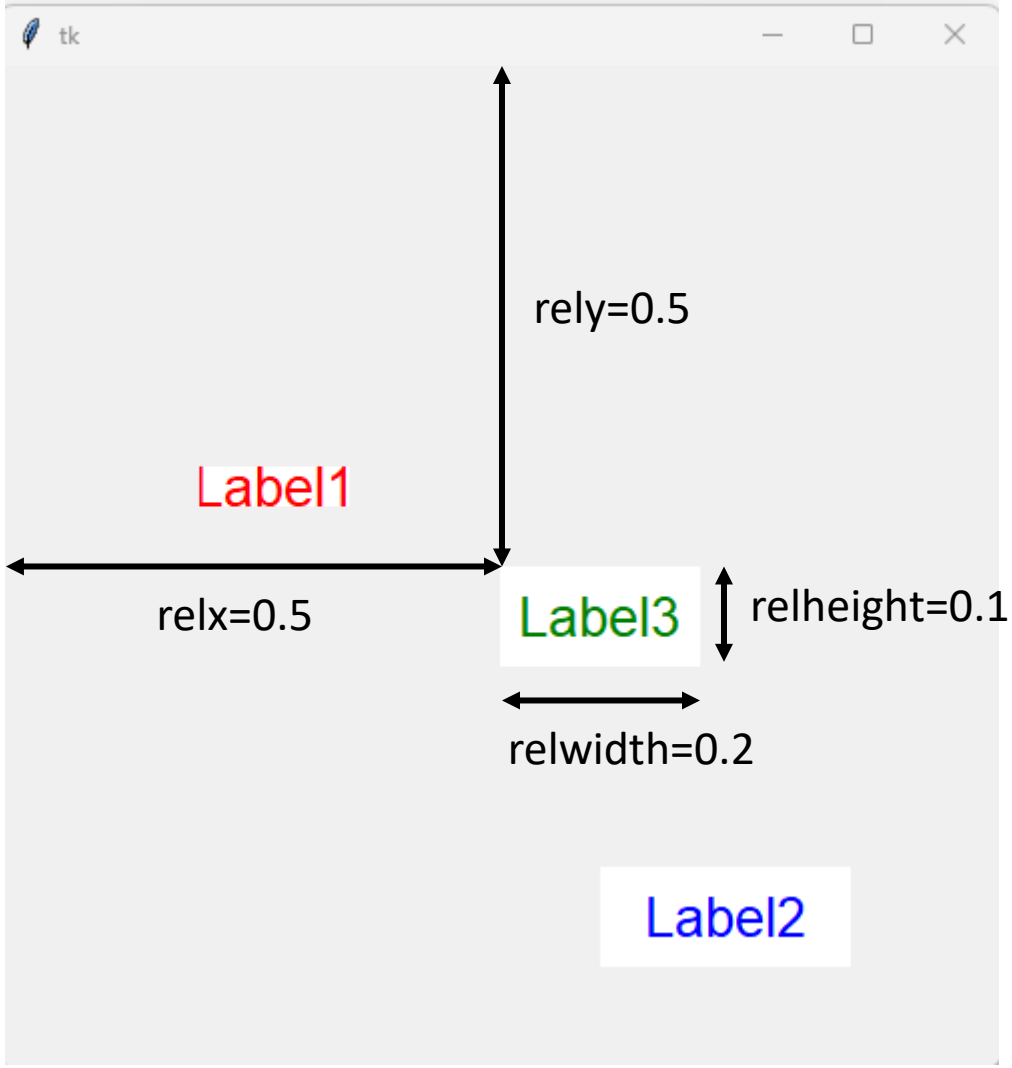
```
label3.place(relwidth=0.2,relheight=0.1,relx=0.5,rely=0.5)
```

- Here we place the top left corner of our label at the center of our window and if the window changes size, it will remain in the center of the afterwards smaller or larger window
- The values we use here are percentages, that means 0.5 stands for 50% of the pixels of the parent in x dimension
- We can also adjust the size by using relwidth and relheight instead of width and height here

Place V

- Relative dimensions of Place:

  - `relwidth=0.2`
  - `relheight=0.1`
  - `relx=0.5`
  - `rely=0.5`



Place VI

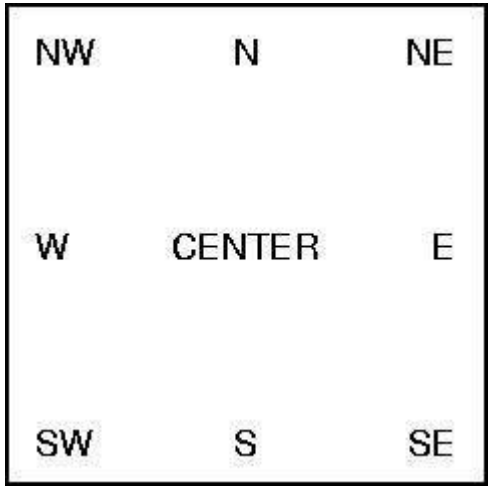
- We are NOT limited to use either relative positioning/sizes or absolute positioning/sizes when working with place
- We can mix these types of usage not only for different widgets but also on one and the same widget
- That means we could define absolute sizes for a widget and than use relx as well as rely to generate a relative positioning of our widget
- Furthermore we can adjust which part of our widget is responsible for the placement
- Per default the coordinates always refer to the top left corner but we can change this by adding anchors to place

Anchors in tkinter [15]

- Tkinter has many different anchors we can use to specify the positioning of:
  - widgets with geometry managers
  - Texts inside of widgets
- Use them as follows:

```
anchor = tk.CENTER

anchor = tk.SW
```

- For place this means the coordinates we specify now refer to the center or the bottom left corner of our widget instead of the default top left corner



Exercise 13: Using place

- Create a GUI that contains at least 6 different Widgets
- Arrange them on your GUI using the place() method
- Try to create some Widgets that are placed dynamically

Opening new Windows

Multiple Windows I

- You learned that the Widget Tk is the window of your GUI
- Another form of this is the Widget Toplevel
- Since this widget also represents a singular window you can use it to create multiple windows at the same time
- For example you can create Pop-Up windows that contains additional functionality, documentation and so on
- To achieve this you need a function that creates a window and starts its mainloop() as shown on the next slide

Multiple Windows II

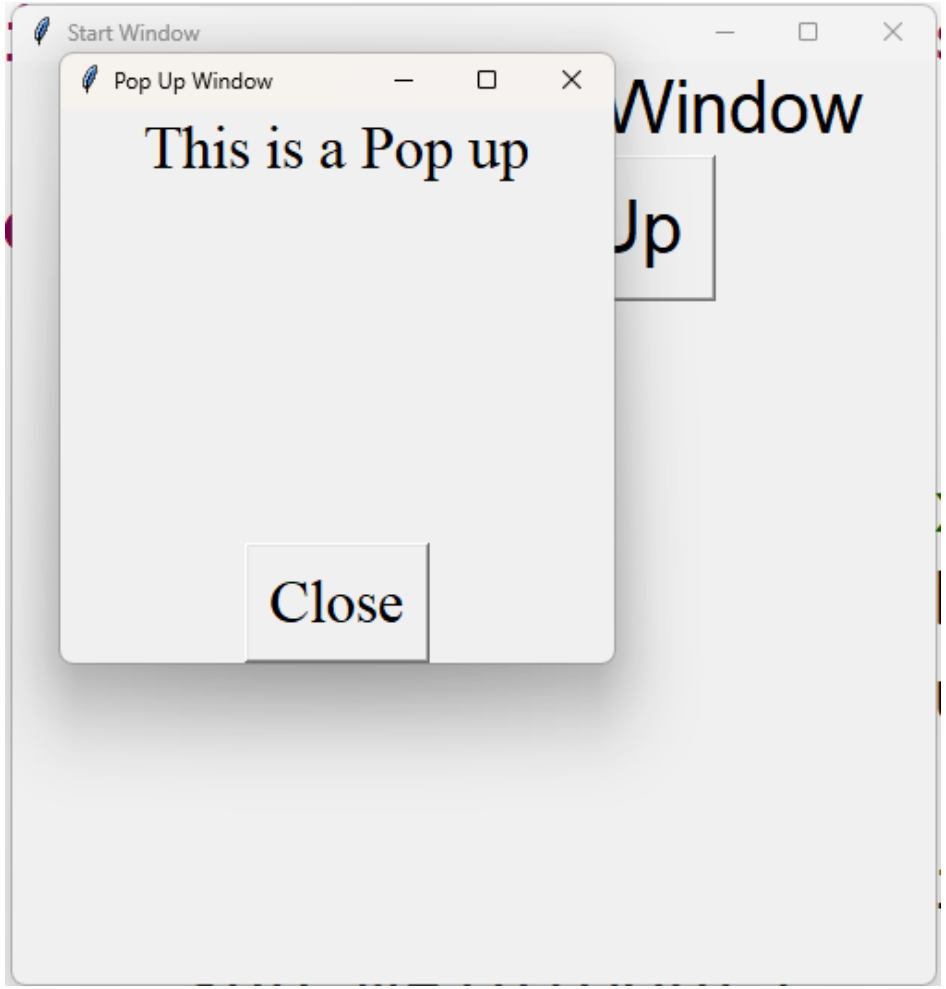
- Down below you can see a function that creates a simple second window
- The second window, called sub, is created by using the tk.Toplevel() function
- Than you can construct and place widgets on it
- The important part is to include the sub.mainloop() line in the function

```
def create_window():
        sub = tk.Toplevel()
        sub.title("Pop Up Window")
        sub.geometry("300x300")
        label_sub=tk.Label(sub,text="This is a Pop up")
        label_sub.pack()
        sub.mainloop()
```

Multiple Windows III

- One Window:

- Two Windows:



Exercise 14: Multiple Windows

- Create a GUI that contains three buttons
- Each button should be associated with a function that opens another Window (3 functions in total)
- Each of these windows should contain at least two different Widgets
- Try to use six different Widgets in total on these additional windows

Lambda

Lambda I [17]

- So far we learned that we can only use functions that don‘t have parameters
- Of course this is rather lack luster
- Therefore we can make use of lambda when constructing our buttons to be able to work with functions that take arguments during the call as well
- Imagine a simple function like this:

```
def ChgT1(Text):
    label1.config(text=Text)
```

- This function has one parameter, a string, and displays this string than on a Label
- Normally we wouldn’t be able to use a function like this with a button

Lambda II [17]

- However by using lambda when constructing our button we are now able not only to associate this function to our Button, but we are also able to hand over an argument to it:

```
button1 = tk.Button(root,text="Click",command=lambda:ChgT1("Hello"))
```

- Here we associate the function ChgT1 with our Button and hand over the String `"Hello"` to it as argument
- Later on when the button is clicked by the user the text of the Label is changed to `"Hello"`

Exercise 15: Using Lambda on GUIs

- Create a GUI that contains at least four Buttons and a Label
- Each Button should be associated with the same function that takes a string argument
  - Use lambda to make this possible
- The function should add the argument to the text of the Label
  - The text in the label gets longer everytime the user clicks on a button

Lambda III [17]

- Another advantage of using lambda is, that you can associate one button with multiple functions at once:

```
button1 = tk.Button(root,text="Click",command=lambda:[ch_bg(),ch_fg()])
```

- The example above shows how to associate one button with two functions at once
- To do so you need to write the function names (ch_bg and ch_fg) inside square brackets and comma separate them
- When the user clicks now on the button the function ch_bg is executed first and ch_fg is executed afterwards
- There is no limit to the amount of functions you can use here

Exercise 16: Lambda with multiple functions

- Create a GUI that allows the user to choose between:
  - two sets of at least four colors
  - a set of at least four different texts
- Create three functions that change the text, the background-color and the foreground color of the same label according to the input of the user
- Associate all three functions to the same button using lambda

forget

forget I [18,19]

- You learned that you can use geometry managers to put widgets onto your GUI
- As you know there are three geometry managers available to you
  - pack
  - grid
  - place
- Sometimes you might want to create a dynamic GUI, that means a GUI where certain widgets are hidden and later on maybe shown again
- tkinter allows for such behavior with three additional methods of your widgets that hide the corresponding widget (next slide)

forget II [18,19]

- When you want to hide a widget you need to use one of the following three methods:
  - pack_forget()
  - grid_forget()
  - place_forget()
- Which method you use depends on the geometry manager you used to put the corresponding widget onto the GUI
- That means when you used pack you need to use pack_forget and so on

Exercise 17: Hide and Show

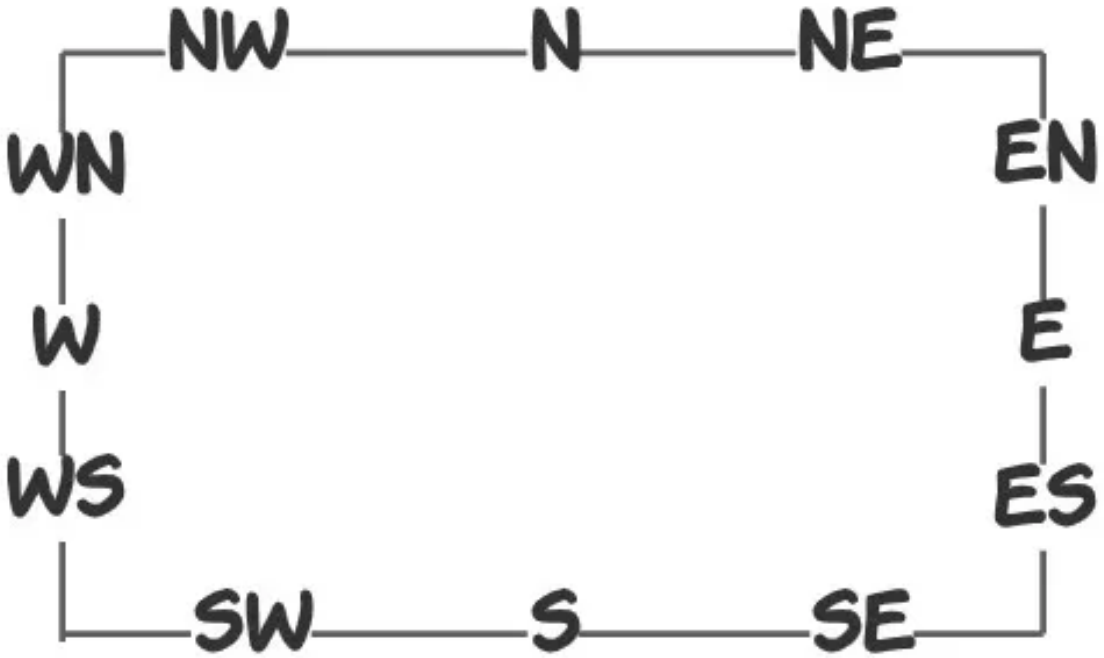
- Create a GUI that contains:
  - four buttons
  - two labels
- Two buttons should allow the user to hide one of the labels
- Two buttons should allow the user to display the labels again
- Use a geometry manager of your choice for this

Other Widgets

LabelFrame [16]

- LabelFrames are Widgets that combine the function of frames to hold other Widgets with the function of Labels to display text
- We can display the text at different positions using the option labelanchor and cardinal directions:

```
label_frame = tk.LabelFrame(root,
text="LAB",labelanchor="s")
```



Exercise 18: LabelFrames

- Create a GUI that contains three different LabelFrames
- Each LabelFrame should contain at least three different Widgets of your choice
- Try to use different positions for the Text of your LabelFrames

OptionMenu I

- With Tkinter you are able do generate a drop down menu on our GUI by using the widget OptionMenu
- We can construct this widget in two different ways, which use the same constructor and in both cases we need to associate a VarVar with it
- The main difference is how we hand over the displayed options in our Menu
- The first way to construct such an OptionMenu is shown below:

```
menu1 = tk.OptionMenu(root,var1,"Option 1","Option 2","Option 3")
```

  - The arguments of OptionMenu are first the parent (root), than the VarVar (var1), than the inputs we want to display as strings (“Option 1”, …) and afterwards all other options

OptionMenu II

- The second way of creating an OptionMenu is by using a list of inputs we defined beforehand instead of writing all inputs into our constructor

```
colors = ["blue", "snow", "grey"]

menu2 = tk.OptionMenu(root, var2, *colors)
```

- Here we use the same constructor and again hand over the parent (root) first, than the VarVar (var2), than the inputs we want to display (*colors)
- The “*” before the list name means that we unpack all elements of the list colors and they are than added comma separated to the constructor

OptionMenu III

- To adjust the look of the OptionMenu you always need to use the config() method of the widget, since the constructor doesn‘t allow you to hand over values for the different options to it:

```
menu1.config(bg="yellow")
```

- Please be aware that this only changes the look of the Menubutton and not the Dropdown menu
- To adjust the Dropdown menu you need the key “menu”:

```
menu1["menu"].config(bg="green")
```

Exercise 19: Options Menu

- Create three Menus
  - Two should contain at least seven colors
  - One should contain at least seven protein names
- Use functions to change the background color, the text color of a Label and the text displayed in that Label corresponding to the chosen Input.

Listbox I [21]

- A listbox displays a list of single-line text items
- It allows users to browse through the list
- Depending on the settings the user can select one or more items from the list
- To construct a Listbox you need the function Listbox as shown below:

```
listbox1 = tk.Listbox(root)
```

Listbox II [21]

- Once you created the Listbox you can add the different options to it, using the insert() method of your Listbox:

```
listbox1.insert(0,"Python")
listbox1.insert(1,"Perl")
```

- Alternatively you can add mutliple options at once adding more arguments to insert()

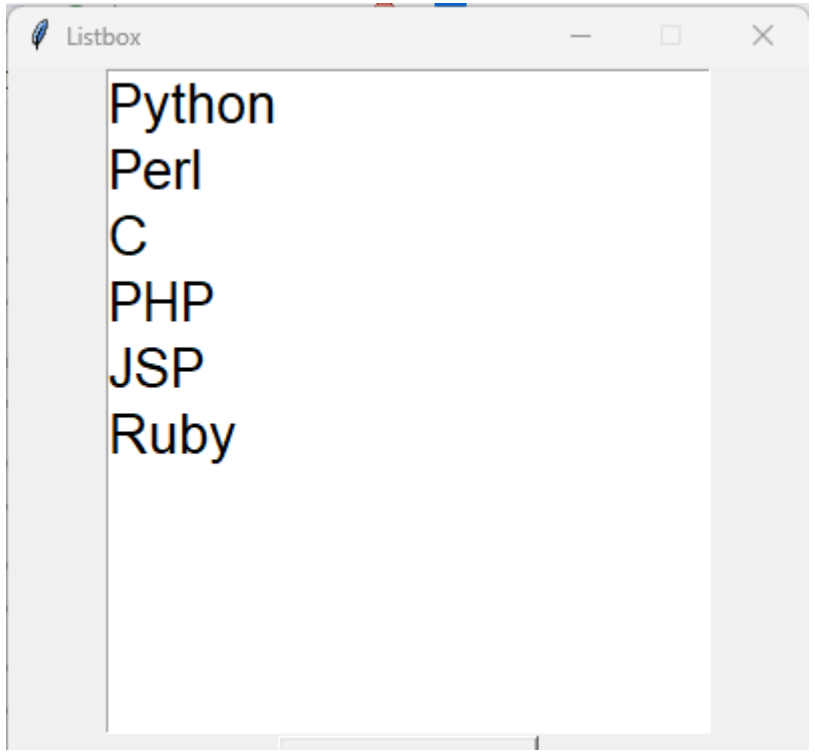
```
listbox1.insert(0,"Python","Perl","C","PHP","JSP","Ruby")
```

- Last but not least you could use a list as well:

```
languages = ["Python","Perl","C","PHP","JSP","Ruby"]
listbox1.insert(0,*languages)
```

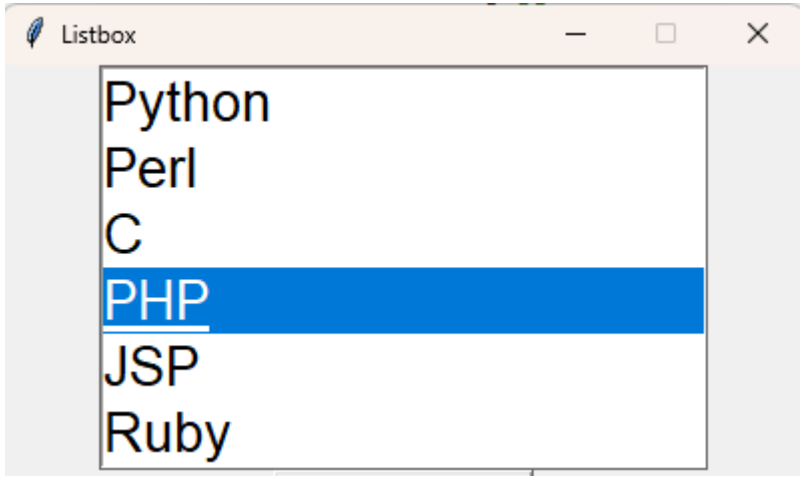
Listbox III [21]

- The result is than as you can see on the right side
- The white part of the GUI is the Listbox containing the different options the user can pick from
- Per default a Listbox will show 10 options, if you want to show more or less you can use the option height with an integer value, e.g. height=6
- Per default the user can pick only one option at a time (next slide)
- For multiple selections you need selectmode="multiple"

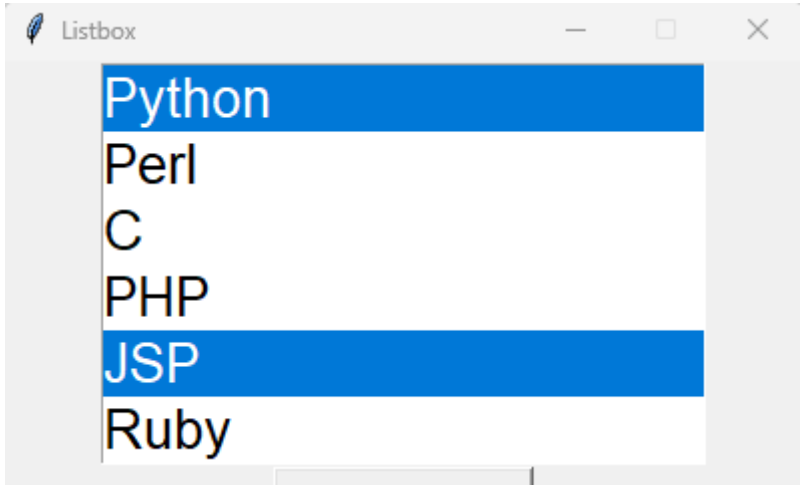


Listbox IV [21]

- Default:



- selectmode="multiple":



Listbox V [21]

- To read out the selected option(s) from the Listbox you need a combination of the methods curselection() and get() of your Listbox
- curselection() gives you back a list of indices (one for each picked option)
- get() allows you to read out the option at a specific index
- A good way to do this is to use a loop construct as shown below:

```
def items_selected():
    selected_indices = listbox1.curselection()
    for i in selected_indices:
        index = listbox1.get(i)
```

Exercise 20: Listing options

- Create a GUI that contains two Listboxes, two Labels and a Button
- One Listbox should allow the user to select multiple options at once
  - Please add the option exportselection=False to both constructors of your Listboxes
- On a click of the Button, display the selected options from the Listboxes in the Labels

Messagebox I [22,23]

- This class of tkinter allows you to open different Messages on your GUI
- Thess Messages are pop-ups that can come in different forms (more details on later slides):
  - Information
  - Error or Warnings
  - Different types of Questions
- Since it’s a class of tkinter you need to import it separately when you want to work with it:

```
from tkinter import messagebox
```

Messagebox II [22,23]

- Each type of Messagebox always uses two arguments:
  1. The title you want to give the appearing Pop-up window
  2. The text/question you want to display on the appearing Pop-up window
- Both arguments need to be strings and are handed over without any keywords

- In total there are 8 different types of messageboxes:
  - showinfo()
  - showerror()
  - showwarning()
  - askquestion()
  - askyesno()
  - askokcancel()
  - askretrycancel()
  - askyesnoycancel()

Messagebox III [22,23]

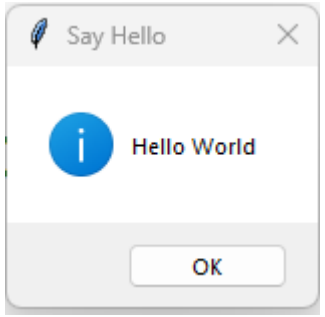
- To use these functions you need to use the point notation with the class messagebox:

```
messagebox.showinfo("Say Hello","Hello World")
```

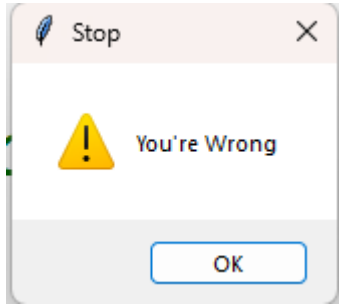
- The example above will create a Messagebox showing an information to the user
- The Pop-Up window will have a title of “Say Hello”
- It will also display the text “Hello World”

Messagebox IV [22,23]

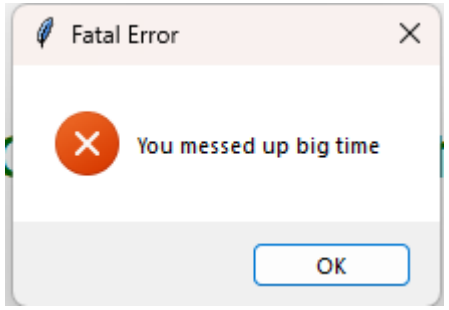
- showinfo()



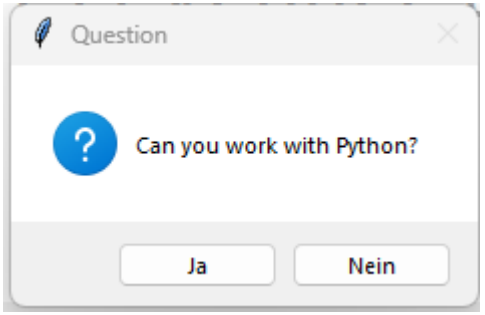
- showwarning()



- showerrror()

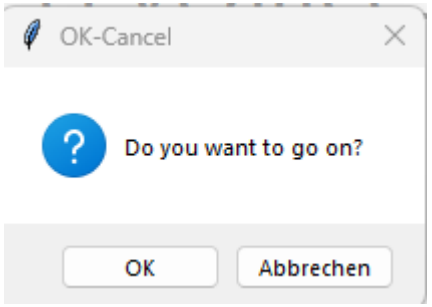


- askquestion()

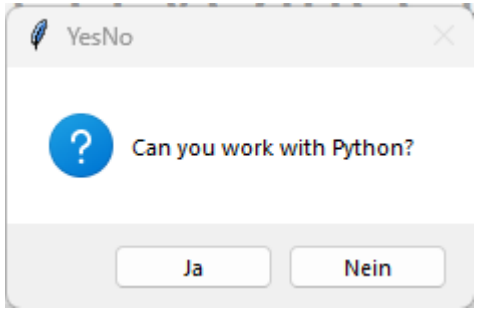


Messagebox V [22,23]

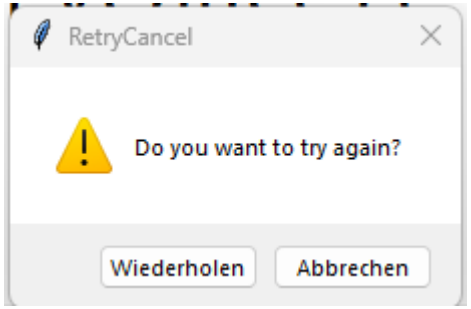
- askokcancel()



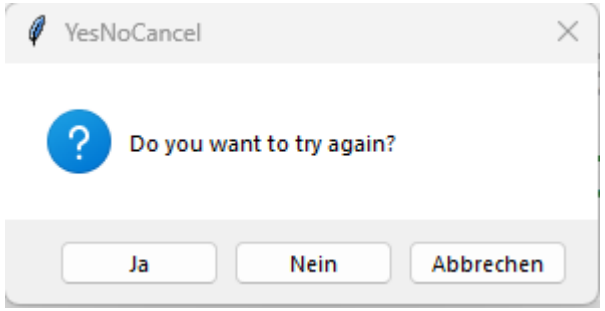
- askyesno()



- askretrycancel()



- askyesnocancel()



Messagebox VI [22,23]

- The „ask“ messageboxes return different values depending on which button the user clicks on in the corresponding Pop-up Window
- Therefore you can assign the returned value to a variable, to use it in if-statements further down the line for example:

```
answer = messagebox.askquestion("Question","Can you work with Python?")
```

- The values you get back differ between the different types of messages (Next Slide)

Messagebox VII [22,23]

- The table below shows you the returned values by the different buttons on the Pop-up windows
- For example the Yes-Button on askquestion returns the string “Yes“, while the No-Button of askyesno returns False

Messagebox	Button1	Button2	Button3
askquestion	Yes = “Yes”	No = “No”	
askokcancel	OK = True	Cancel = False	
askyesno	Yes = True	No = False	
askretrycancel	Retry = True	Cancel = False	
askyesnocancel	Yes = True	No = False	Cancel = None

Exercise 21: Messages

- Create a GUI that contains three Radiobuttons, a normal button and a label
- On the click of the button different messages should be displayed
  - an information
  - a warning
  - an error
- Depending on which Radiobutton was activated
- The function should furthermore change the text on the label depending on the picked option

Menu I [24]

- Menus are the important part of any GUI
- A common use of menus is to provide convenient access to various operations such as:
  - saving or opening a file
  - quitting a program
  - manipulating data
- Toplevel menus are displayed just under the title bar of the root or any other windows

Menu II [24]

- To add a Menu to your Window you need to create it first:

```
mainmenu = tk.Menu(root)
```

- Than you can add the different options with the add_command method

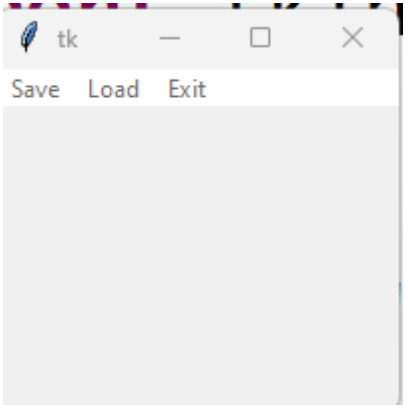
```
mainmenu.add_command(label="Save",command=save)

mainmenu.add_command(label="Load",command=load)

mainmenu.add_command(label="Exit",command=root.destroy)
```

- Last but not least you need to combine the window with the Menu

```
root.config(menu=mainmenu)
```

Menu III [24]

- The finished GUI than looks as shown on the right side
- You can see that a Top-Menu was added to the window with the different Options Save, Load and Exit



Menu IV [24]

- Often you don‘t want just simple buttons in your Top-Menu, instead you want to add dropdown menus there
- To do so you need to create additional menus and use you Top-Menu as parent of them:

```
toolmenu = tk.Menu(mainmenu)
```

- Now you can add the inner options again with the add_command method

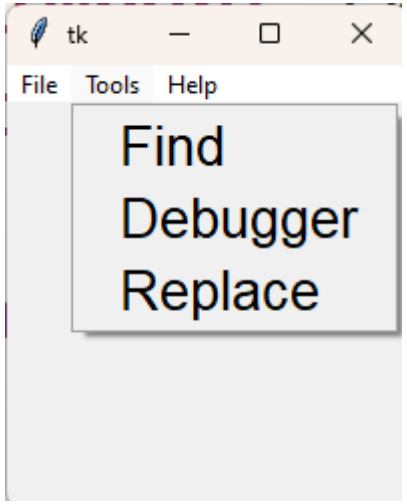
```
toolmenu.add_command(label="Find",command=emptyfunc)
toolmenu.add_command(label="Debugger",command=emptyfunc)
toolmenu.add_command(label="Replace",command=emptyfunc)
```

Menu V [24]

- Once your Inner menu is finished you can add it to the Top-Menu using the add_cascade() method:

```
mainmenu.add_cascade(label="Tools",menu=toolmenu)
```

- The result is than a Top-Menu containing a dropdown menu as shown on the right side
- Depending on your needs you could add even Dropdown menus to your inner Menu, e.g. associated with Find



Exercise 22: Professional GUI

- Create GUI that contains a Top Menu
- The Menu should contain at least 4 options, e.g. File, Edit, View & Help
- The corresponding options should contain Dropdown Menus with at least 3 options each
- Optional: Try to add a Dropdown Menu to the Dropdown Menu

Exercise Guessing Game

Exercise Snake

Exercise Treasure Hunt

Definitions

Definitions I

- **GUI** is the abbreviation for Graphical User Interface
- The main body of a GUI is the **Window** that functions as **Root**
- All graphical elements on a GUI are called **Widgets**
- In tkinter a **Parent** is a widget on which we can place other widgets
- All widgets are adjustable with the help of **Options** we define in the constructor of our widget
- A **Label** is a widget that can be used to display text
- The way text looks like is defined be its **Font**
- A **Button** is a widget that the user can click on to execute a certain function
- Variables associated with widgets are called **VarVars**

Definitions II

- Simple text fields that allow the user to type something in, we than can read out and work with, are called **Entries**
- Dropdown Menus can be created on your GUI by using the Widget **OptionMenu**
- Widgets that allow only True or False inputs are called **Checkbuttons**
- Another form of multiple choice input by grouping together multiple widgets of the same type can be achieved by using **Radiobuttons**
- You can get numerical inputs in form of a Scrollbar using **Scales**
- Numerical inputs or inputs from a pre-defined list of values can be generated via **Spinboxes**

Definitions III

- Methods used to position Widgets on the GUI are called **Geometry Managers**
- **Frames** are invisible widgets that can be a parent for other widgets
- The geometry manager **pack()** is only able to place widgets to the four sides left, right, top or bottom
- The geometry manager **grid()** uses the concept of rows and columns for positioning widgets
- the geometry manager **place()** uses either pixel-exactly placements or relative placements in regards to the parent
- Widgets that combine the function of frames and labels are called **LabelFrames**

Literature I

1. https://www.activestate.com/blog/top-10-python-gui-frameworks-compared/ (Last updated: 09.08.2022, Last visited: 12.04.2023)
2. https://www.csse.canterbury.ac.nz/greg.ewing/python_gui/ (Last updated: 25.03.2017, Last visited: 12.04.2023)
3. https://wiki.python.org/moin/Wax (Last updated: 13.03.2014, Last visited: 12.04.2023)
4. https://pyforms.readthedocs.io/en/v3.0/ (Last visited: 12.04.2023)
5. https://pypi.org/project/PySimpleGUI/ (Last updated: 11.10.2022, Last visited: 12.04.2023)
6. https://www.libavg.de/site/ (Last visited: 12.04.2023)
7. https://kivy.org/ (Last visited: 12.04.2023)
8. https://pypi.org/project/PySide2/ (Last updated: 14.01.2022, Last visited: 12.04.2023)
9. https://www.wxpython.org/ (Last updated: 07.08.2022, Last visited: 12.04.2023)
10. https://riverbankcomputing.com/software/pyqt/intro (Last visited: 12.04.2023)
11. https://docs.python.org/3/library/tkinter.html (Last updated: 12.04.2023, Last visited: 12.04.2023)

Literature II

12. https://en.wikipedia.org/wiki/Tkinter (Last updated: 14.03.2023, Last visited: 12.04.2023)
13. https://www.tutorialspoint.com/python/tk_fonts.htm (Last visited: 12.04.2023)
14. https://www.pythontutorial.net/tkinter/tkinter-grid/ (Version from 2021, Last visited: 27.2.2022)
15. https://www.tutorialspoint.com/python/tk_anchors.htm (Last visited : 07.06.2022)
16. https://coderslegacy.com/python/tkinter-labelframe/ (Last visited : 02.06.2022)
17. https://www.tutorialspoint.com/tkinter-button-commands-with-lambda-in-python (Last visited: 14.04.2023)
18. https://www.geeksforgeeks.org/python-forget_pack-and-forget_grid-method-in-tkinter/ (Last edited: 29.11.2021, Last visited: 30.05.2024)
19. https://www.geeksforgeeks.org/place_forget-method-using-tkinter-in-python/ (Last edited: 01.08.2020, Last visited: 30.05.2024)
20. https://www.analyticsvidhya.com/blog/2024/01/ways-to-convert-python-scripts-to-exe-files/ (Last edited: 01.04.2024, Last visited: 30.05.2024)
21. https://tk-tutorial.readthedocs.io/en/latest/listbox/listbox.html (Last edited: 2020, Last visited: 02.09.2024)

Literature III

22. https://www.geeksforgeeks.org/python-tkinter-messagebox-widget/ (Last edited: 26.03.2020, Last visited: 02.09.2024)
23. https://docs.python.org/3.9/library/tkinter.messagebox.html (Last edited: 01.05.2024, Last visited: 02.09.2024)
24. https://www.pythontutorial.net/tkinter/tkinter-menu/ (Last edited: 2023, Last visited: 02.09.2024)